

ui.checkbox 全面详解（基于 NiceGUI 文档）

ui.checkbox 是 NiceGUI 中用于实现布尔值选择（勾选 / 取消勾选）的核心交互组件，基于 Quasar 的 QCheckbox 组件实现。它支持基础勾选交互、状态监听、数据绑定、样式定制等能力，适用于表单勾选、功能启用 / 禁用、多选列表等需要二值选择的场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QCheckbox 组件，继承其成熟的勾选交互逻辑、样式体系和无障碍支持。
- 核心功能：仅支持布尔值状态（选中 `True` / 未选中 `False`），通过点击复选框或标签触发状态切换，支持初始状态配置和状态变化监听。
- 交互特性：点击复选框本身或其关联的文本标签，均可切换选中状态，提升操作便捷性。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
text	str	复选框右侧的文本标签	-
value	bool	初始选中状态（ <code>True</code> 为选中， <code>False</code> 为未选中）	<code>False</code>
on_change	Callable	状态变化时触发的回调函数（ <code>e.value</code> 为当前选中状态）	-

二、基础使用示例

1. 最简用法（文本标签 + 状态联动）

展示基础文本标签、初始状态配置，以及与其他组件的状态联动：

```
from nicegui import ui

# 基础复选框（带文本标签，初始未选中）
checkbox = ui.checkbox('显示隐藏内容', value=False)

# 联动标签：复选框选中时显示，未选中时隐藏
ui.label('被控制的隐藏内容').bind_visibility_from(checkbox, 'value')

ui.run()
```

效果：

- 复选框右侧显示文本“显示隐藏内容”，初始未勾选；
- 勾选复选框后，下方标签显示；取消勾选后，标签隐藏，实现状态联动。

2. 状态变化监听（用户交互 + 手动修改）

区分“用户点击触发”和“代码手动修改”两种状态变化场景，展示不同事件绑定方式：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. on_change: 监听所有状态变化（用户点击+代码修改）
    ui.label('on_change（监听所有状态变化）：')
    c1 = ui.checkbox(on_change=lambda e: ui.notify(f'状态: {"选中" if e.value else "未选中"}'))
    ui.button('切换状态', on_click=lambda: c1.set_value(not c1.value))

    # 2. on('click'): 仅监听用户点击触发的状态变化
    ui.label('on("click")（仅监听用户点击）：')
    c2 = ui.checkbox().on('click', lambda e: ui.notify(f'用户点击后状态: {"选中" if e.sender.value else "未选中"}'))
    ui.button('切换状态', on_click=lambda: c2.set_value(not c2.value))

ui.run()
```

效果：

- 第一个复选框：无论用户点击还是按钮触发状态变化，都会触发 `on_change` 回调；
- 第二个复选框：仅用户点击时触发 `click` 回调，按钮手动修改状态时不触发，实现交互场景区分。

三、核心功能与进阶用法

1. 数据绑定（双向同步）

通过 `bind_value` 实现复选框与数据对象的双向绑定，数据变化时组件自动更新，反之亦然：

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.agree_terms = False  # 初始未同意条款

state = AppState()

with ui.column().classes('gap-4'):
    # 复选框与数据对象绑定
    ui.checkbox('我已阅读并同意用户协议', value=state.agree_terms).bind_value(state, 'agree_terms')

    # 显示绑定数据的实时状态
    ui.label().bind_text_from(
        state, 'agree_terms',
        lambda x: f'协议状态: {"已同意" if x else "未同意"}'
    )
```

```
# 手动修改数据对象，组件自动同步
ui.button('一键同意', on_click=lambda: setattr(state, 'agree_terms', True)).classes('mt-2')

ui.run()
```

效果：

- 点击复选框切换同意状态，`state.agree_terms` 自动同步；
- 点击“一键同意”按钮修改数据对象，复选框自动勾选，实现双向绑定同步。

2. 样式定制（颜色、尺寸、标签位置）

通过 `props` 和 `classes` 定制复选框的颜色、尺寸、标签位置等样式：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 自定义选中颜色（红色）
    ui.checkbox('红色选中', value=True).props('color=red')

    # 2. 紧凑模式+大尺寸
    ui.checkbox('紧凑+大尺寸', dense=True).props('size=lg')

    # 3. 标签在左侧（默认在右侧）
    ui.checkbox('标签在左', label_position='left').props('label=left')

    # 4. 自定义文本颜色和间距
    ui.checkbox('自定义文本样式', classes='text-blue-600 gap-2') # gap-2 增大复选框与文本间距

ui.run()
```

效果：

- 四个复选框分别展示不同样式，支持颜色、尺寸、标签位置、文本样式的定制，适配不同界面设计需求。

3. 禁用状态与批量控制

通过 `set_disabled` 禁用复选框，或实现多个复选框的批量控制（全选 / 取消全选）：

```
from nicegui import ui

# 全选复选框
select_all = ui.checkbox('全选')

# 待选列表
options = ['选项1', '选项2', '选项3', '选项4']
checkboxes = []
with ui.column().classes('ml-6 mt-2 gap-1'):
    for opt in options:
        cb = ui.checkbox(opt)
        checkboxes.append(cb)
```

```
# 全选逻辑：选中全选框时勾选所有选项，取消时取消所有选项
def on_select_all_change(e):
    for cb in checkboxes:
        cb.set_value(e.value)

select_all.on_change(on_select_all_change)

# 单个选项变化时更新全选状态
def on_option_change():
    all_checked = all(cb.value for cb in checkboxes)
    select_all.set_value(all_checked)

for cb in checkboxes:
    cb.on_change(on_option_change)

# 禁用所有选项的按钮
ui.button('禁用所有选项', on_click=lambda: [cb.set_disabled(True) for cb in
checkboxes])).classes('mt-4')

ui.run()
```

效果：

- 勾选“全选”复选框，所有子选项自动勾选；取消“全选”，所有子选项自动取消；
- 子选项全部勾选时，“全选”自动勾选；部分勾选时，“全选”保持未勾选；
- 点击“禁用所有选项”按钮，所有复选框变为禁用状态，无法点击切换。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
enabled	BindableProperty	是否启用（可绑定数据动态控制，禁用后不可交互）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
text	BindableProperty	文本标签（可绑定数据动态修改）
value	BindableProperty	选中状态（True / False，可绑定数据）
visible	BindableProperty	是否可见（可绑定数据）
props	Props[Self]	Quasar 特性属性（如 color 选中颜色、size 尺寸）

2. 关键方法

(1) 状态控制

- `disable()`: 禁用组件 (不可交互, 视觉灰度)
- `enable()`: 启用组件
- `set_disabled(value: bool)`: 设置启用状态 (`True` 禁用, `False` 启用)
- `set_visibility(visible: bool)`: 设置组件可见性

(2) 属性修改

- `set_value(value: bool)`: 动态修改选中状态 (`True` 选中, `False` 未选中, 触发 `on_change` 事件)
- `set_text(text: str)`: 动态修改文本标签
- `update()`: 修改属性后, 调用此方法刷新界面 (绑定数据时无需手动调用)

(3) 事件与绑定

- `on_change(callback)`: 绑定状态变化事件 (`e.value` 为当前选中状态, 覆盖所有触发方式)
- `on(type: str, handler)`: 订阅任意 DOM 事件 (如 `click` 仅监听用户点击, `mousedown` 等)
- `bind_value(target_object, target_name)`: 双向绑定选中状态到目标对象的属性
- `bind_text_from(target_object, target_name)`: 单向绑定文本标签从目标对象

(4) 其他实用方法

- `tooltip(text: str)`: 为组件添加悬停提示
- `delete()`: 彻底删除组件
- `mark(*markers)`: 添加标记 (用于测试或元素查询)

五、使用场景与注意事项

1. 适用场景

- 表单勾选: 如用户协议同意、隐私政策授权、表单选项勾选等。
- 功能开关: 如界面功能启用 / 禁用、显示 / 隐藏特定内容等。
- 多选列表: 如数据筛选条件、批量操作选择项等。

2. 关键注意事项

- 状态类型限制: 组件 `value` 仅支持布尔值 (`True/False`), 不支持其他类型 (如字符串、数字)。
- 事件触发差异: `on_change` 监听所有状态变化 (用户点击 + 代码修改), `on('click')` 仅监听用户点击触发的变化, 需根据场景选择。
- 标签交互: 文本标签与复选框联动, 点击标签即可切换状态, 无需单独绑定事件。
- 版本兼容性: `html_id` 属性仅在 2.16.0+ 版本支持, `bind_enabled` 的 `strict` 参数在 3.0.0+ 版本支持, 使用时需确认版本。
- 与 `ui.switch` 的区别: `ui.checkbox` 适用于多选场景或需要明确文本标签的二值选择; `ui.switch` 更侧重功

能开关的视觉表达，样式更紧凑。

六、进阶示例：带条件启用的复选框组

实现基于前置条件的复选框启用逻辑（如勾选“同意基础条款”后，才允许勾选“高级功能授权”）：

```
from nicegui import ui

# 前置条件复选框
base_agree = ui.checkbox('同意基础服务条款', value=False)

# 依赖前置条件的复选框（初始禁用）
advanced_agree = ui.checkbox('授权高级功能使用', disabled=True)

# 前置条件变化时，启用/禁用高级功能复选框
def on_base_agree_change(e):
    advanced_agree.set_disabled(not e.value)
    # 取消前置条件时，自动取消高级功能授权
    if not e.value:
        advanced_agree.set_value(False)

base_agree.on_change(on_base_agree_change)

# 显示当前授权状态
ui.label().bind_text_from(
    base_agree, 'value',
    lambda x: f'基础条款: {"已同意" if x else "未同意"}'
)
ui.label().bind_text_from(
    advanced_agree, 'value',
    lambda x: f'高级功能: {"已授权" if x else "未授权"}'
)

ui.run()
```

效果：

- 未勾选“同意基础服务条款”时，“授权高级功能使用”复选框禁用，无法勾选；
- 勾选基础条款后，高级功能复选框启用，可正常勾选 / 取消；
- 取消基础条款勾选时，高级功能自动取消勾选并禁用，确保授权逻辑一致性。